Data Transparency

Individual text documents are no longer the sole/primary means of disseminating academic scholarship: developments influencing the emergence of "Executable Research Objects"

Replication Crisis Difficulty in reproducing both older, influential studies and new ones

Data Transparency becomes a priority Wider access to raw data can — hopefully! — help clarify why replication is a problem

MIBBI and friends Minimum Information for Biological and Biomedical Investigations

"FAIR" Findable, Accessible, Interoperable, Reusable

Open-Access Data Sets Raw data files are usually shared according to different copyright and availability protocols than academic/scientific publications themselves

Research Object Specifications Research Object "Bundle"; Executable Research Objects

“Replication Crisis”

- This phrase began to be used in the 2010s
- Some “failures to reproduce” derive from statistical or computational errors
- Controlled experimentation is particularly questionable vis-à-vis social sciences and psychology
- “Chaos effect”: subtle differences in setup and parameters can yield significant differences in results

Data Transparency Becomes a Priority

- Publishers may require authors to share raw data or else explain specifically why it is not possible to do so
- Funding agencies may require data transparency as a precondition for grantees (e.g., the Bill and Melinda Gates Foundation)
- "Data Availability" or "Supplemental Materials" sections are now common on publication landing pages

Authors however often confuse tables or summaries with actual data sharing — i.e., publishing raw data sets (not just a statistical overview)

MIBBI and Other Minimum Information Standards

Minimum Information for Biological and Biomedical Investigations

- Standard for describing experiments, lab, equipment, and data acquisition
- Employs structured metadata in lieu of text descriptions
- MIBBI encompasses almost 40 specific formats, such as:
  - Minimum Information About a Microarray Experiment (MIAME)
  - Minimum Information About a Proteomic Experiment (MIAPE)
  - Minimal Information Required In the Annotation of Models (MIRIAM)
  - Minimum Information About a Simulation Experiment (MIASE)
  - Minimum Information about a Flow Cytometry Experiment (MIFlowCyt)
  - Minimum Information about a Functional Genomics Experiment (MIAFGE)
  - Core Information for Metabolomics Reporting (CIMR)
  - Proteomics Identifications Database (PRIDE)

"Findable, Accessible, Interoperable, Reusable"

- FAIRdata: Standard for how data is organized and presented
- FAIRsharing: Standard for how data is published and deposited
- Accessibility: Beyond just raw data, minimizing reliance on special software or computing environmets — prefer open-source
- Reusability: Effort should be made to help future scientists analyze the raw data — share code to give future researchers a head start

Data Sets

- At a minimum, data sets are *citeable* (independently referenced) collections of one or more raw data files
- Data sets are often published outside any paywall, with a more permissive licence than accompanying papers — not an ideal solution
- For FAIRsharing, raw data sets should be accompanied by computer code — it usually requires special programming to deserialize, manipulate, and do useful things with the raw data
- Don’t assume users can load raw data with external software (e.g., CSV often doesn’t work well just viewed as a spreadsheet)

Executable Research Objects

- A way to achieve FAIR data transparency: data, code, and multimedia bundled togethers
- Research Object "Bundles" follow a constrained folder layout and required JSON manifest, with other recommended metadata
- Executable Research Objects are less formally defined, but with an emphasis on code sharing

Other related terms are "Research Compendia", "Story Maps" (in ArcGIS), "Data Infrastructure Building Blocks" (DIBBs), "Open Data Cube" (ODC), or "The Whole Tale" (a DIBBS initiative)

Publications Within Research Objects

Or so we might predict:

In the future ...

- Executable Research Objects will encompass text documents as well as code and data
- Readers may first discover publications as PDF files via web search — or from other documents/bibliographies — but they might later read the text in a Research Object context
- Executable Research Objects can embed PDF viewers tailored to their specific data sets

Which means ...

- Manuscripts can be cross-referenced with other Research Object content
- PDF Viewers can be programmed with extra code to implement cross-reference logic
- Page coordinates can be noted via `\pdfsavepos` and friends

What's "Cross-Reference Logic"?

Generic Examples

- Adding options to context menus in proximity of specific document contents
- Reading and acting upon hyperlinks (maybe something other than just scrolling to or loading the link target)
- Reading data within PDF annotations or comment boxes
- Adding GUI controls around page margins

Specific Examples

- If a graphic shows a video frame, play the video
- If video frames are geotagged, open a digital map display
- If a video records a simulation, open windows for the simulation graphics themselves, or to view initial parameters and/or implementation code
- Graphics specific to individual disciplines: e.g., chemical formulae may be linked with molecular-visualization files and windows
- Multimedia Cross-referencing: Consider a video showing an environmental remediation project, due to contamination by several toxic chemicals — potentially launching video, cartographic, and molecular display windows

Text Mining

Helping researchers find publications relevant to their work — in theory

So Many Publications For most topics, too many peer-reviewed papers for researchers to consider each one individually. Can Text Mining and/or Natural Language Processing tools help?

Modeling Rhetoric The most useful papers for a researchers will typically not only be *about* some topic. They will, more specifically, *endorse*, *critique*, or neutrally *evaluate* an argument or theory

Thematic Connections If a paper cites another, what discursive role does this reference serve? To support a claim the current author makes or repeats from elsewhere; to indicate an influential precedent in the field; to identify an early example where a theory or terminology was introduced; to direct readers toward a lengthier or more historically significant discussion of a topic; to single out the referenced work as exceptionally clear, influential, or innovative; or to initiate a critical analysis of the work?

Across Disciplines Computational tools might discover connections between subject-areas that researchers miss on their own

Case Study: "CORD-19"

- Over 400,000 full-text open-access papers about Covid-19, curated by Allen Institute for AI (AI2)
- Roughly 1/3 of the documents had JATS available; otherwise PDF extraction was used
- AI2 openly acknowledged the limitations of PDF extraction and issued a "Call to Action" encouraging publishers to develop better text encoding/metadata
- Converted all papers to S2ORC-JSON in the hope that developers could program text-mining tools facilitating COVID research

Limitations of CORD-19

- Transcription errors for technical terms/formulae
- Documents were only modeled down to the paragraph level (not individual sentences)
- Limited support for semantic annotations and controlled vocabularies
- Inconsistent text encoding means that potential connections between disparate documents are not algorithmically recognized
- Machine-readable sources (XML, LaTeX, etc.) from publishers' internal workflows often unavailable

But Let's Not Get Too Locked In on Text Mining

There are other use cases for machine-readable text encoding

- Improving upon internal PDF search:
  - Load or embed JATS (or similar structures) alongside PDFs — then redirect PDF search to these files
  - Avoid problems due to ligatures, hyphenation, etc.
- More granular searching inside PDFs:
  - Restrict queries to footnotes, quotations, (sub)section titles, etc.
  - Correctly handle ellipses; quotation-emendations; citation data
  - Allow systematic use of semantic annotations
- Extend the scope of Full-Text Query Languages:
  - In conjunction with reverse-index (vector) databases (as current)
  - Searching "meta" indexes (pooling books' print/curated indexes)
  - Embedded in PDF Viewers or Executable Research Objects
  - As a standalone tool for authors and editors

Sentence Objects

SENTENCES ARE THE BASIC UNITS OF DISCOURSE

- **Written Natural Language** XML, JSON, or other markup is merely an approximate serialization of textual and discursive structures
- **Sentence Nesting/Divergence Points** Sentences usually have a simple sequential organization, but not always (cf. footnotes, block quotes, enumerations)
- **Object-Oriented Implementations** Textual objects should be conceptualized and, as much as possible, exposed to computer code as class-instances for Object-Oriented languages such as Java or C++. Markup should be intended for serialization, not to provide the definitive structure of documents for querying and other computational requirements.
- **Multiple Scales** Keyphrases and annotations provide a sub-sentence level of organization. Paragraphs and (sub)sections provide a super-sentence level.

Sentence Objects and Query Results

sentences.sdi (~/gits/pnp-dev/presentations/dev/

File Edit View Search Tools Documents Help

```
--- Abstract/start


--- Sentence/start
id: 0

--- Sentence//end
id: 0
r#  216  9  50
g.
|   @l(-%>) +19/6/1
|   +23/6/5:0=23/6/5
|   +33/6/15:0=33/6/15
|   +38/6/20:0=38/6/20
.   @l() -38/6/20
p: .
t.
|  In data sharing and publishing
|  contexts such as Executable Research Objects,
|  natural-language documents coexist with raw
.  data files and, potentially, multimedia resources


--- Sentence/switch
id: 1
r#  217  9  51


--- Sentence//end
id: 1
r#  318  11  53
p: .
```

sentences-result.txt (~/git

File Edit View Search Tools Documents Help

```
>find PDF :in $Sentences :save to %-result.txt
  :report-width 85
===



# 26  = PDF ->
|  Unfortunately, the most common format for d
|  is flawed from a computational perspective,
|  natural language often differs from how hum
|  (e.g., when seeing it rendered in a PDF vie

# 27  = PDF ->
|  Such details as ligatures, headers/footers,
|  artifacts in text extraction (or scraping)

# 28  = PDF ->
|  To clarify these remarks: PDF code librarie
|  to obtain text from one page and/or window

# 31  = PDF ->
|  The most straightforward contrast would be
|  occur, such as unexpected space characters
|  fonts (small spaces may be present in the i
|  readers, but methods like Page::text have n
|  characters)

# 33  = PDF ->
|  In addition to problems with ordinary langu
|  of special character strings such as organi
|  not use a consistent mechanism for notating
|  textual elements (or, at least, structured
```

Text Elements as Structured Objects

- Sentence objects themselves (the core of an effective object system for written language)
- Paragraps
- PDF pages
- Proper names (different conventions of order, initials, alphabetization, middle versus compound)
- Place names (can be geotagged)
- Legal statutes
- Sentence-like landmarks (definitions, statements of theorems/lemmas, axioms)
- Chemical formulae
- Citations (bibliographic reference, plus visible number or string, plus page range or other locator) — bidirectional links to bibliography
- Quotations (need to model ellipses, emendations, paragraph breaks, sources) — show source in separate window?
- Index locators (visible Para ids? Mark contentent matching index headings?)
- Para id subdivision (e.g., inside/around block quotes)
- Figures, images, and other graphics content — possibly interactive

Para Ids and Sentence Boundaries

Grammar overall, in terms of "Structural Empiricism":

i Viewed through the lens of structural empiricism, Cognitive Grammar is therefore a family of models of a linguistic system. Like any other scientific theory, it features theoretical entities and processes (cognitive domains, constructional schemas, compositional paths, subjectification, etc.) and empirical substructures isomorphic (in a weaker version: similar) to appearances of observable phenomena.[27, p. 18]

a He immediately avows "It is somewhat unclear what should count as observable phenomena in linguistics", but situates such uncertainty in a larger philosophical context. In particular, *most* scientific theories entertain taxonomies and analyses of a domain of entities encompassing both public observables and "hypothesized" posits that — for one of several possible reasons — cannot be "seen". Empiricism is *structural* insofar as such "unobservables" are taken seriously because they contribute to persuasive explanations of *observables'* behavior, with structured models unifying the seen and the hidden into a rational package. Sometimes unseen posits are deemed just as real as everything else, merely beyond our capabilities of (instrument-enhanced) observation, at least for now. Other phenomena (like "dark energy") are left more open-ended, possibly subsumed under such ontological realism or possibly artifacts of some more complex process (e.g., "phonons", which behave like "particles" of sound, but overlooks the fact that many useful scientific models involve clearly counterfactual scenarios [whose] role is not merely heuristic or pedagogical.[27, p. 32]

He then analyzes how a

ii fictional models in natural sciences can be accounted for in philosophical terms. In … structural empiricism, isolated systems, frictionless pendulums, and perfect liquids are treated just like any other theoretical objects, except their theoreticity does not follow from unobservability, as is the case with electrons and electromagnetic fields, but from their fictional character. [A] structural empiricist is not committed to believing in the actual existence of real-world objects corresponding to theoretical concepts, but only to the empirical adequacy of the theory featuring these concepts.[27, p. 33]

A reasonable paraphrase vis-à-vis constructed sentences is that, for purposes of expositing semantic or syntactic claims, the universe of linguistic objects is broader than merely the totality of sentences humans have in fact created. Ad-hoc hypotheticals also belong to that domain if they serve and fit within model-building regimes. b

I would put it thus: what makes a sentence an object in the domain of entities spotlighted by linguistics is not the fact of its being empirically an example of real language-use, but rather how upon that sentence we can apply ob-

Continuation Marks: a, b, c; i, ii, iii; left/right column

just as real as everything else, merely beyond our capabilities of (instrument-enhanced) observation, at least for now. Other phenomena (like “dark energy”) are left more open-ended, possibly subsumed under such ontological realism or possibly artifacts of some more complex process (e.g., “phonons”, which behave like “particles” of sound, but that’s largely a metaphor [33]). Natural selection or “design” is not an object, but rather an encompassing system wherein adaptation drives evolution. Potentially “dark” matter and energy will ultimately be given a similarly indirect and holistic cosmological interpretation, rather than deemed to refer to a specific kind of field or particle.

ated. Ad-hoc hypotheticals also belong to that domain if they serve and fit within model-building regimes.

I would put it thus: what makes a sentence an object in the domain of entities spotlighted by linguistics is not the 45
fact of its being empirically an example of real language-use, but rather how upon that sentence we can apply observations, analyses, annotations, etc., whose ordinary ground is real-world speech-events. We can, for instance, note syntactic, morphological, or lexical patterns, and point out where pragmatic interpretation is needed. Constructed sentences may or may not conform to English grammar, but at least the question of whether they do so

45 is well-posed. Even *imagined* speech is grammatical (or not). We can label syntactic categories; track pronoun-antecedent pairings and singular or plural alignment; observe how inflections indicate verb-tense; and etc.: minimal operations in the syntactic and semantic realms that apply equally whether samples are real or hypothetical.

My earlier subway-platform speculation was similarly
46 about hypothetical circumstances. I have caught New York subways in real life, but that’s beside the point: we are primed to learn from experience, but in a manner

two verbs. The sentence is not merely notating observable 48a
details: choice of words can encode information about the speaker’s epistemic starting-point. This phenomenon is nicely illustrated in (15) and (16), constructed or not.

But let’s step back. For sake of discussion, I will assume
(15) and (16) occur in contexts where listeners know the 49
speaker saw John firsthand (the evidence is direct observation, not hearsay). Implicit in a communicative claim to *describe* something witnessed is having been in a perceptual orientation to *see* it. The sentences then encode

Paragraph Subdivisions and Index Locators

Sentence Examples ★

[1] My brother has been married for 10 years, but in many ways he’s still a bachelor. 1 2

[2] All of the eligible bachelors in this town are married. 1 2

[3] Everyone sang along to three songs. 1 3

[4] All appeals are heard by a panel of three judges. 1 3

[5] All New Yorkers live in one of five boroughs. 1 3

[6] All New Yorkers complain about how long it takes to commute to New York City. 1 3

[7] In Ariel’s picture of Beatrice, she looks sick. 1 3

[8] In Ariel’s picture of Beatrice, she snapped the lens just as two cardinals flew by. 1 3

[9] We operated near the bank. 1 3

[10] On the mantle was a wooden bat. 1 3

[11] Every band performed three songs. 2 7

[12] Everyone sang along to some song or another. 2 7

[13] This election is slipping away from the Democrats. 2 8

[14] Student after student came by to complain about the reading assignments. 2 8

[15] John poured water onto the stove. 8 48

[16] John spilled water onto the stove. 8 48

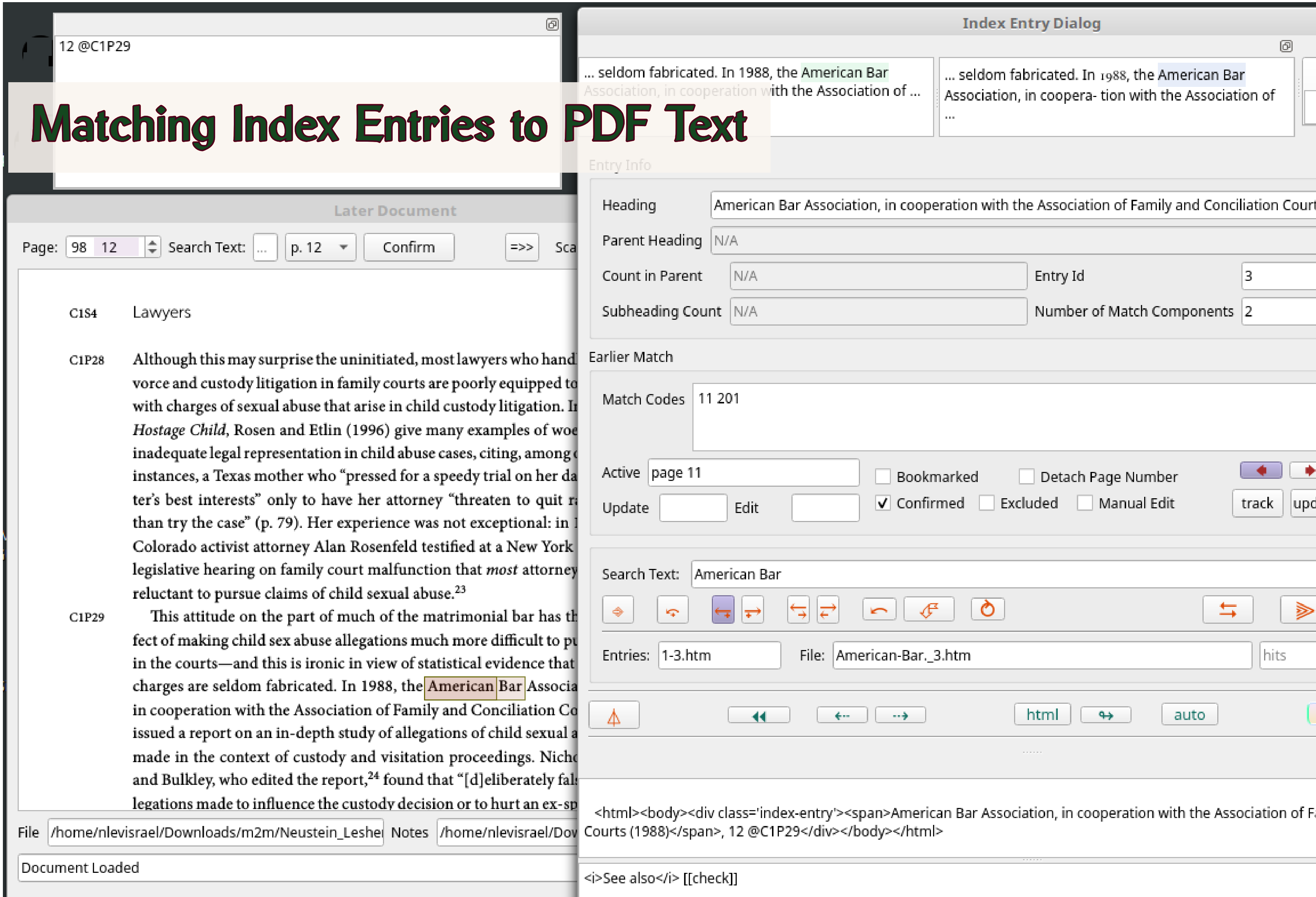
[17] I put Mom’s chocolate brownies in this red tin box, take some! 9 60

[18] I’m heading out to the store. 13 80

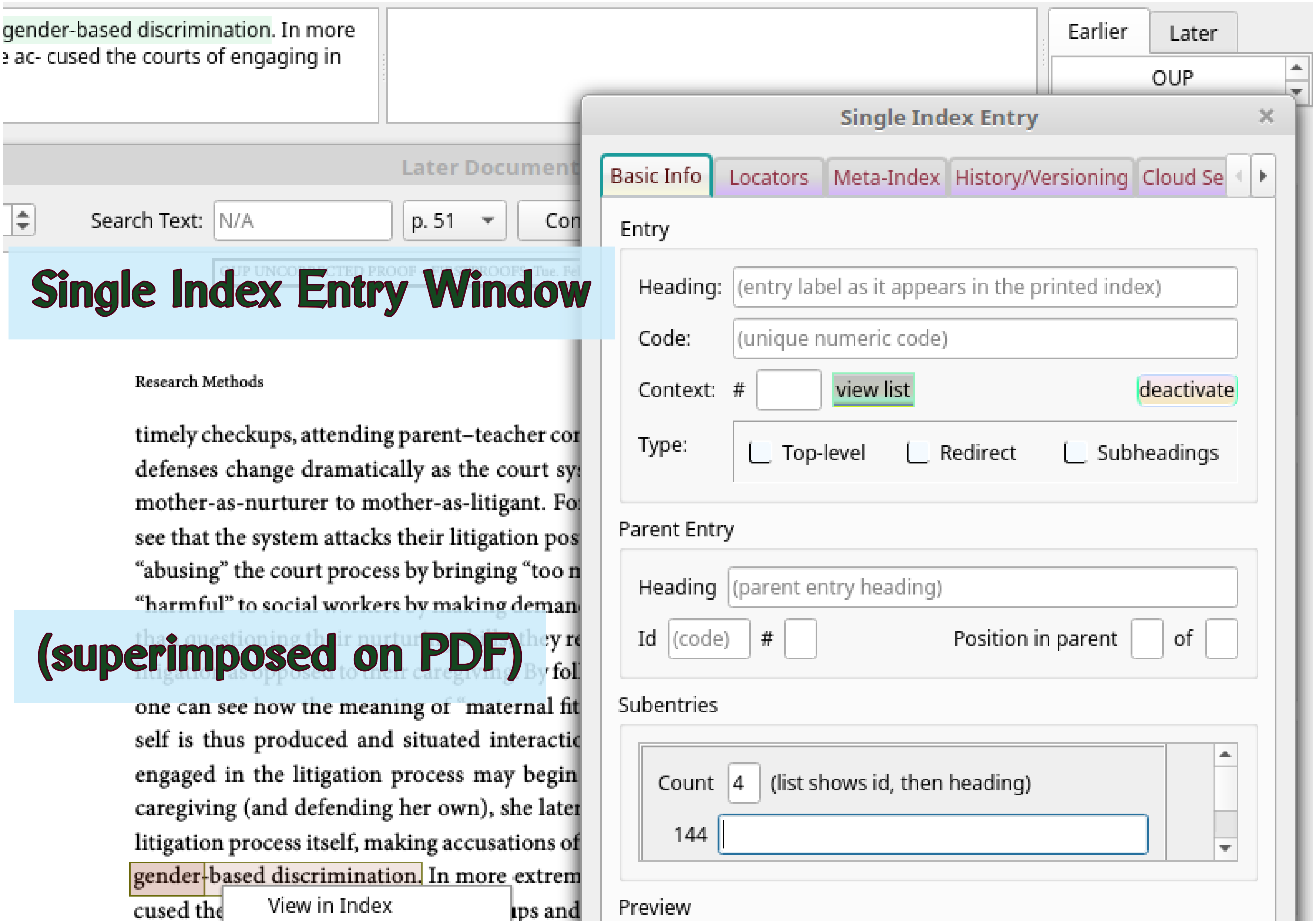
Index †

GUI Controls for (Print) Indexing

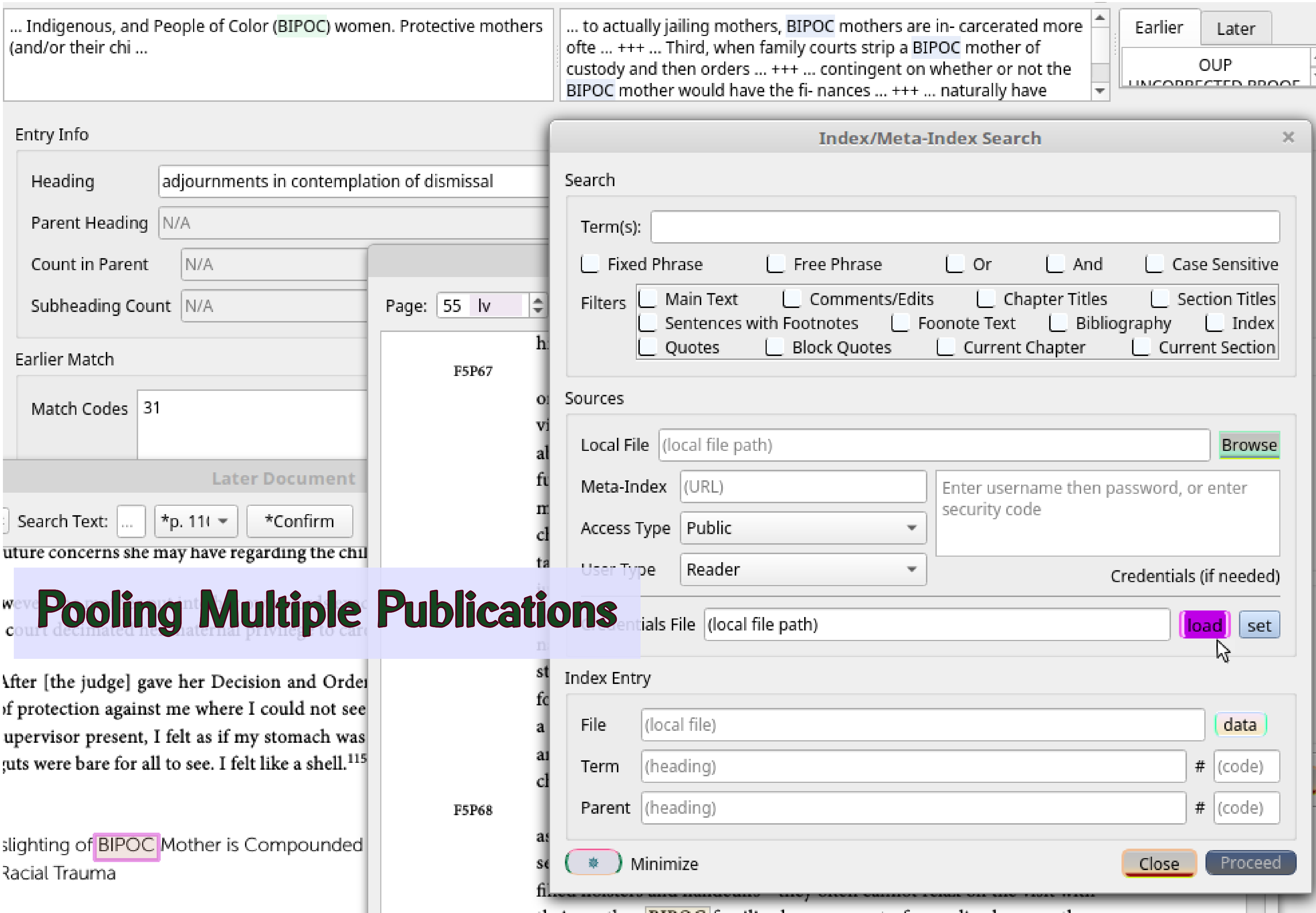
Matching Index Entries to PDF Text



GUI Window for Single Index Entry

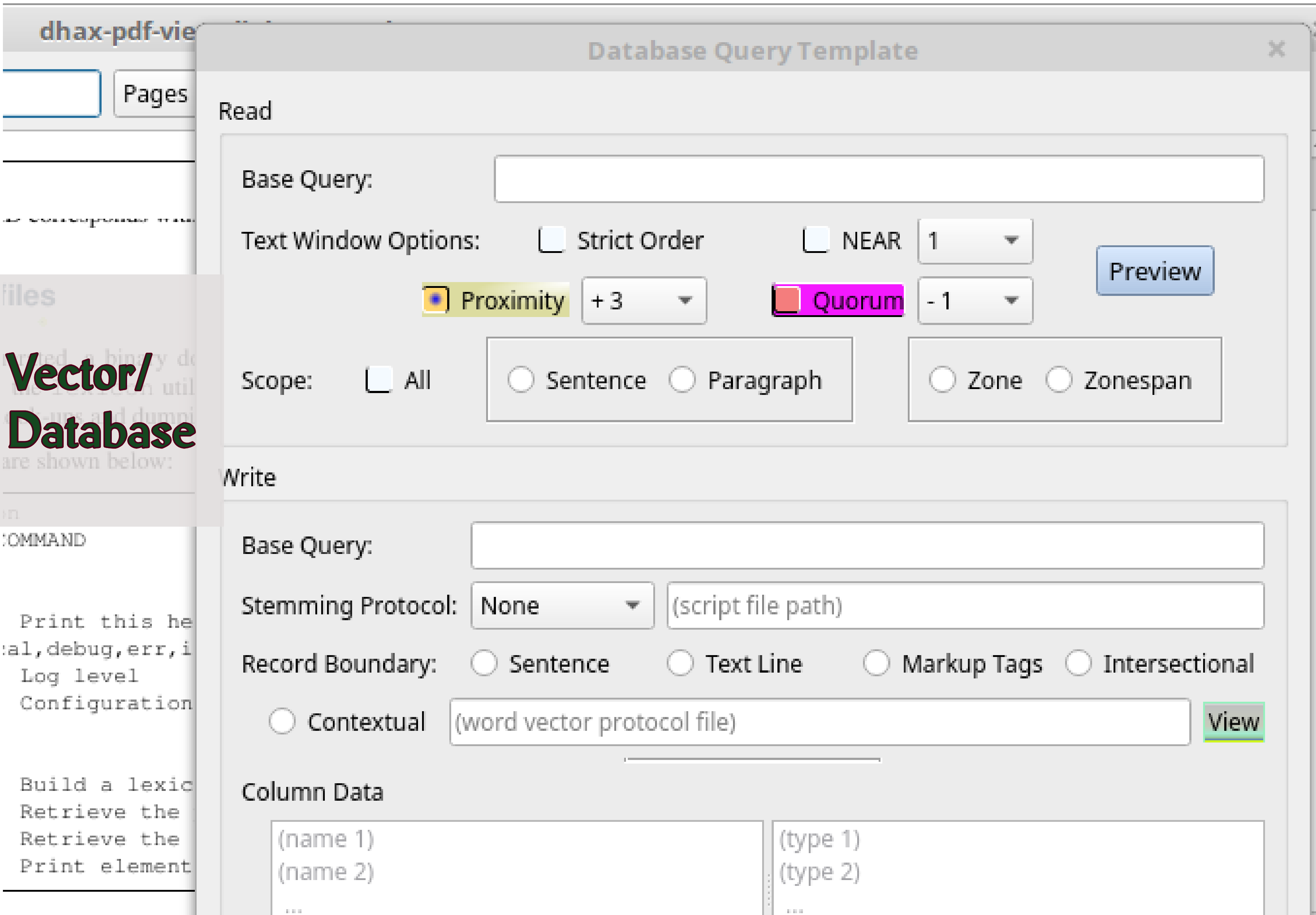


Meta-Index Search



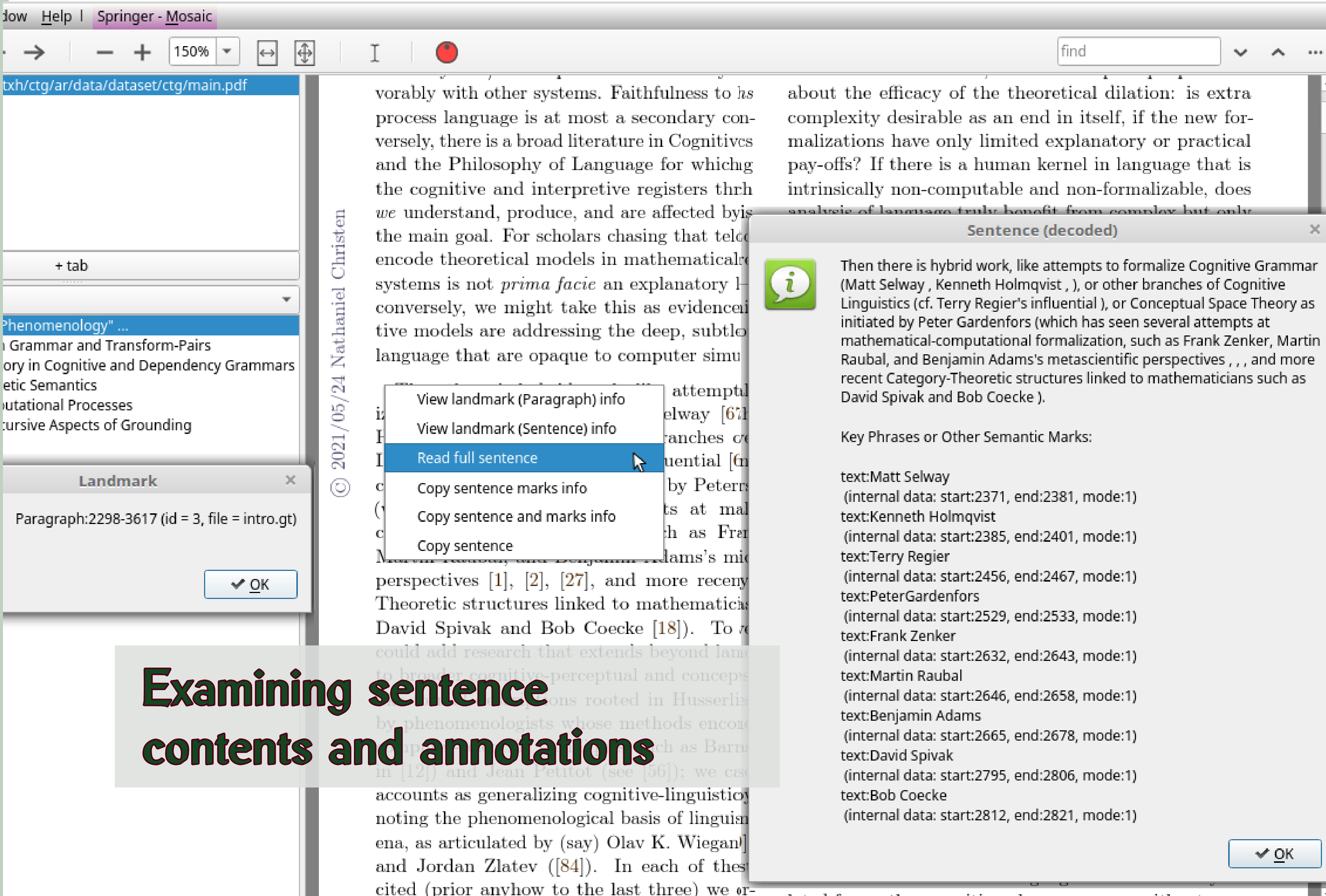
Pooling Multiple Publications

Reverse-Index Searches



Vector/ Database

Viewing Sentence and Annotation Boundaries



Examining sentence contents and annotations

Data Objects for Executable Research Objects

So, Research Objects' internal information space covers two main kinds of objects (in the sense of class-type values)

Document-related Objects

- Sentence objects and other text elements
- Objects related to manuscript versions, changes, provenance
- PDF annotations and comment boxes
- Index entries and locators (headings, subentries, "*see*" and "*see also*" redirection)

Dataset Objects

- Individual records/observations as class instances
- Collections of records/observations (tables, lists, associative arrays, queues)
- Individual fields/measurements as class members
- Objects encapsulating structured data such as MIBBI protocols
- Multimedia assets and GUI controls that render them

Cross-References (again)

Cross-referencing between Text Objects and Data Objects

- Column names, units of measurements, scales, etc., can be matched to text strings
- PDF page areas can be given interactive capabilities to launch GUI controls oriented toward specific object classes in a data set
- Text/Code cross-references: data types, procedures, preconditions, source files
- Algorithms and analyses: computational processes discussed in the text and implemented in Research Object code

GUI Indexing for Research Objects

- Context Menus: each menu has a list of options, including label text and actions/callbacks
- Drop-Down Lists or "Combo" boxes: often items can be associated with a controlled vocabulary
- Mouse gestures: where different gestures are possible and what they mean
  - Drag
  - Key-press modifiers
  - Cursor icons
- Separate windows: top-level, dialog, wizard
- Model-View Controller pattern: each GUI class is correlated with a data class — component's state at any one time shows one object
- User manuals
  - Interactive documentation (as a special GUI mode)
  - Functionality (remember, windows can get reorganized)
- Key-combo extensions — bind keypress combinations to mini-scrips (*cf.* Emacs LISP)

Computational Support for Text/Data Integration

Scripting and Query Languages

- Full text query built around "Sentence Objects"
  - Query results can take the form of sentence lists
  - Sentences provide a match context (e.g., find two words in the same sentence)
  - Restrict matches to the beginning of a sentence, or the interior
  - Find the first sentence in a chapter (or section, etc.) where a match occurs
  - Find annotated text-to-data cross-reference points, in PDFs and text encoding
- Scripting for Research Object data
  - Deserialize raw data files
  - Customize GUIs for individual data sets
  - Implement algorithms, analyses, data types
  - Automate unit tests
  - Define workflows, simulations, data-processing steps or protocols
  - Provide standalone applications as Executable Research Object entry points
  - Customize PDF viewers for individual Executable Research Objects

Using Scripts to Enhance Research Object Presentations

- Avoid relying on external deserialization libaries
- Design by Contract (DbC): Explicit preconditions and programming assumptions
- Explicit scales, coordinate systems, units of measurement
- Use dataset code to clarify theoretical background, data acquisition protocols, data models, and formulae/subroutines documented via procedure implementations

The Problem with Deserialization

Generic or Custom Formats

Some discipline-specific data-encoding formats are based on generic languages such as XML or JSON. Others use domain-specific markup or representation formats. .

Navigation and Traversal

For those based on generic formats like XML, deserialization code must still navigate through a DOM tree in order to extract the specific data fields needed in each context.

Custom Parsers

For discipline-specific representations, an additional step is input files to create tree or graph structures to begin with.

Library Maintenance

In either case, deserializing data encoded to domain-specific standards requires dedicated computer code. Consequently, individual organizations and academic departments often take responsibility for maintaining libraries geared to deserializing specific data formats, typically those associated with their particular research specialization.

Language Stacks for Research Objects

Compiler and Virtual Machine architecture for Executable Research Objects

Support both query and scripting languages There are use cases for both forms of computer language in Research Object contexts

Minimize External Dependencies As much as possible, langauge compilers and runtimes should be built directly from source files internal to data sets

Customizable It should not take extensive extra coding to add syntactic and runtime features designed specifically for individual data sets

Focus Language Design around Research and FAIR Sharing Special languages inside data sets are not intended to be general-purpose scripting languages. Instead, their syntax and semantics should be focused on the goal of documenting research methods, theories, and data models

Provide Compiler and VM Support for Desired Language Features Design by Contract, unit-decorated types, coordinate-system-specific types, multiple-dispatch, MVC architecture, and other patterns for semantically detailed languages require bottom-up support vis-à-vis instruction sets and runtime state

Parsing for an Embedded Compiler

Useful design patterns for parsers oriented to self-contained compiler stacks

Avoid relying on external tools such as Lex, Yacc, Bison, LLVM, etc.

- Grammars should be context-sensitive: they might have one or more notions of “context” to support language extensions and expressive syntax
- Individual rules are regular expression objects coupled with callback handlers, and optionally contextual filters: we are not generating code *for* a parser, we are just implement rule-lists that are tried in sequence — anchored at the current “cursor” position — until one regex matches
- Rule callbacks can modify parser state/contexts, but in general would emit code for a “first-stage” Virtual Machine
- This preliminary VM then calls instructions for a code generator that produces “bytecode” for the main VM
- In both contexts “bytecode” is actually calls to class methods, and can be printed in human-readable fashion (although compressed bytecode can employ single bytes per instruction)

Rules and Contexts in Parsing Code

chtr-grammar.cpp | # | ChTR_Grammar::init(ChT... | chasm-tr-p

ser [presentations]
parser.pro
up
parser
-tr-parser.pri
group
rs
me/nlevisrael/gits/pnp-dev/pre
codegen
grammar
chtr-grammar.h
chtr-graph-build.h
chtr-node-factory.h
chtr-parse-context.h
chtr-parser.h
chtr-pregraph.h
chtr-document.h
chtr-source-file.h
es
me/nlevisrael/gits/pnp-dev/pre
codegen
grammar
chtr-grammar.cpp
chtr-graph-build.cpp

```
ChTR_Parse_Context& parse_context = graph_build.parse_context();

add_rule(source_context,
  "carrier-declaration",
  ", (?<symbol> \\S+) (?<tween> \\s+) (?<tx> [^,;*&)\\]] \\S*)"
  ,[&]
{
 pregraph.reenter_statement_level(p.position_pair());

 QString sym = p.matched("symbol");
 QString tween = p.matched("tween");
 QString tx = p.matched("tx");
 pregraph.prepare_carrier_declaration(sym, tween, tx);
});

add_rule(flags_all_(parse_context ,active_qu                      t,
  "query-lambda-token-expecting-another",
  "(?<token> [^\\s;]+) (?<post> ;+) .spaces.? "
  ,[&]
{
 pregraph.query_lambda_token(p.matched("token"), p.matched("post"));
});

add_rule(flags_all_(parse_context ,active_query_lambda), run_call_context,
  "query-lambda-token",
  "(?<token> \\S+)"
  ,[&]
{
 pregraph.query_lambda_token(p.matched("token"));
});
```

“Outer” contexts can be grouped such that one triggers another

Rules are regular-expression strings plus callbacks

“Inner” contexts are combinable boolean flags

The Idea of "Channels"

- Group together input and/or output parameters with related (maybe special-purpose) semantic profiles
- Language extensions can be implemented via special types of channels (e.g., supporting *queries* inside *scripting* formats by building channels that model query parameters)
- Back-door channels can implement proof-checks and other DbC features
- Channels provide convenient search-points for static analysis or runtime monitors

Building Channels Programmatically or From VM Code

```
@fn --sf-- ;.

init-source-file-lexical-scope ;.
source-file-index $ 1 ;.

 .; statement-level declaration ;
load-type-u1 ;.
declare-lexical-typed-symbol $ x

 .; statement-level pin ;.
single-init-pin $ x ;.

load-value-literal $ 5 ;.
resolve-pins ;.

 .; statement ;.
statement-line-number $ 5 ;.
init-new-ghost-scope ;.
push-carrier-deque ;.

new-call-package ;.
add-new-channel $ proc ;.
load-proc-name $ prn ;.

 .; expression ;.
statement-line-number $ 5 ;.
push-carrier-deque ;.

new-call-package ;.
gen-return-channels ;.
add-new-channel $ proc ;.
load-proc-name $ + ;.

add-new-channel $ lambda ;.
load-carrier-symbol-lxs $ x ;.
load-unsigned-literal-int $ 6 ;.
```

```
s]
s]
ations]
ri
pri
ot.pri
ot-both
l-root-both.
l-root-both-
uild-root-bo
i
'gits/pnp-de
ns]
s]
s]
esentations
```

chasm-lib-console/main.cpp* # testqs1(QString, fl...

```cpp
void run_testqs1(Chasm_Runtime* csr)
{
 Chasm_Call_Package* ccp = csr->new_call_package();
 ccp->add_new_channel("lambda");

 QString a1 = "A1";
 float a2 = 12345.6789;
 u1 a3 = 135;

 Chasm_Carrier cc1 = csr->gen_carrier<QString>(&a1);
 Chasm_Carrier cc2 = csr->gen_carrier<r8>(&a2);
 Chasm_Carrier cc3 = csr->gen_carrier<u1>(&a3);

 ccp->add_carriers({cc1,cc2,cc3});

 ccp->add_new_channel("retv");
 Chasm_Carrier cc0 = csr->gen_carrier<u1>(csr->Retvalue._ul
 ccp->add_carrier(cc0);

 csr->evaluate(ccp, 301361_cfc, &testqs1, &cc0);
 u1 result = cc0.value<u1>();
 qDebug() << "r = " << result;
}

void run_testqs1n(Chasm_Runtime* csr)
{
 Chasm_Call_Package* ccp = csr->new_call_package();
 ccp->add_new_channel("lambda");

 QString a1 = "Test";
 u4 a2 = 33;
 u1 a3 = 111;

 Chasm_Carrier cc1 = csr->gen_carrier<QString>(&a1);
 Chasm_Carrier cc2 = csr->gen_carrier<u4>(&a2);
 Chasm_Carrier cc3 = csr->gen_carrier<u1>(&a3);
```

Hidden or "Back Door" Channels

Real and hypothetical examples

- The implicit "message receiver" or "*this*" object — compilers will match procedure names to class methods even if *this* is not explicitly present among arguments
- Memory buffer to be initialized via Return Value Optimization
- "Proof" objects to prevent functions with Design by Contract stipulations from being called unless preconditions are known to have been tested
- Symbols captured by a closure
- Exceptions propagated outside a *try*/*catch* block

Interface Design Principles for the Research Object Context

What are some design patterns that should be encouraged?

- **Unify Pass-by-Reference Initialization and Pointer Return** Called procedures agnostic as to the source of initialized memory buffers/objects (whether pre-initialized reference arguments, delegating ownership of heap-allocated memory, or Return Value Optimization)
- **Programmer Control of Variable Lifetimes** Lifetimes can be associated with different lexical scopes or other variables'
- **"Overload by Contract"** Programmers supply different versions of a procedure depending on whether contract-tests are known at compile time to have been evaluated
- **Easy Foreign Function Interface** Calling native-compiled procedures should be possible with simple registration fuctions
- **Self-Documenting** Given $y = f(x)$, is y being initialized or re-initialized? Does f have side-effects? Was y default constructed?
- We can guarantee *const*. Can we *guarantee* to modify as with an lvalue?

Diamond Open Access

According to the Diamond (DOA) model, publications have neither Article Processing Charges (APCs) nor paywalls

- **Authors Retain Copyright** No fees for either authors or readers. Authors can share their work in any form (PDF, XML, HTML, etc.)
- **Unify Publications and Data Sets** Once manuscripts have the same licences as associated Research Objects, both PDF and machine-readable text representations of the written documents can be freely distributed within open-access data sets.
  - **This makes PDF customization and embedded full-text search possible**
- **Give Authors Greater Control** To compensate for missing APC revenue, publishers can reduce their workload per publication by delegating more responsibility to authors. This can be to authors' benefit and can streamline the document-prep process.
- **Option to Retain Outside Help** If authors do want assistance with their manuscript, funds otherwise marked for APCs can instead be used to hire specialists who can perform work needed to prepare a manuscript for publications. Instead of just doing copy or digital editing, these specialists can take on the more complex tasks implied by the Executable Research Object paradigm, such as implementing dataset code and preparing raw data files.

Translational Science and Translational Medicine

Emphasizing the social benefits of scientific breakthroughs

A Rigorous Theory Although most science seeks positive impact, specialists in Translational Science/Medicine have developed detailed models of the stages research undertakes during the translational process

Research Object Foundations The code and protocols internal to research objects can lay the basis for on-site deployments and their digital/computational requirements

Emphasizing Community Outreach For community-oriented and/or public health initiatives, scientists may need to partner with social workers, language interpreters, experts in local culture, and potentially legal or economic advisers.

Scholarship "en situ" Observations from those directly working with community members and translational-science stakeholders may be just as valuable as less hands-on academic researchers. Publishers should encourage a pipeline for books and articles from the on-site, community/nonprofit context parallel to the ecosystem of institutional academic research.

Science/Humanities Integration

Software Language Engineering has a humanities dimension

- Source code does not "communicate with" computers, but there is still a linguistic dimension to how programmers' intentions are translated to code.
- The processing pipeline for compilers and Virtual Machines is arguably closer to linguistics than to mathematics

Executable Research Objects and the humanities

Translation to social benefit relies on cultural knowledge specific to humanities fields

- While raw data itself depends on narrow scientific disciplines, the organization and dissemination of data packages implicates humanities themes such as intertextuality and Philosophy of Science
- Humanities scholars may be especially sensitive to the nuances of written documents, rhetoric/argumentation, and sentence object modeling

Text Mining

Thanks for Listening

This slide deck can be seen at
https://scignscape.github.io/PNP/documents/JATS-CON-2026-slides.pdf

A parallel set of 41 "textovers" is at
https://scignscape.github.io/PNP/documents/JATS-CON-2026-slides-textover.pdf

The original paper is https://scignscape.github.io/PNP/documents/A-perspective-from-compiler-theory.pdf

Author's email: osrworkshops@gmail.com